Implementação de Grafos em C++

Universidade Estadual do Oeste do Paraná

3ª Avaliação - Estrutura de Dados

Professor MSc. Leonardo Medeiros

31 de outubro de 2026

Vinícius Almeida Hermes
vinicius.hermes@unioeste.br

Introdução

Este trabalho consiste na documentação de duas implementações da estrutura de grafos, para obtenção da nota pertinente à terceira avaliação da disciplina de Estrutura de Dados, ministrada pelo professor MSc. Leonardo Medeiros. Com isso, estão documentadas as implementações de grafos por matriz de adjacência e por lista de adjacência, ambas utilizando a linguagem C++.

Grafo por Matriz de Adjacência

Uma das formas de representar um grafo é utilizando uma matriz de adjacência. Dado um grafo G com n vértices, podemos representá-lo em uma matriz n x n $A(G) = [a_{ij}]$. Cada índice a_{ij} da matriz armazena o peso da aresta que conecta a_i a a_j. Se $a_{ij} = 0$, não há aresta ligando estes dois vértices [1].

Implementação

Definição da classe

Descrição: A classe armazena o peso de cada aresta em uma matriz de tamanho máximo de 100 x 100. Cada linha representa o vértice de origem e cada coluna representa o vértice de destino. Uma posição com valor zero indica que não existe aresta entre os respectivos vértices. Já uma posição com valor diferente de zero representa o peso da aresta. Os vértices são identificados por índices no intervalo [0, numero_vertices - 1].

Arquivo: `grafo_matriz.hpp`

```
class Grafo
{
private:
    TipoPeso Matriz[MAX_VERTICES][MAX_VERTICES];
    int N_Vertices;
    int N_Arestas;

public:
    explicit Grafo(int numero_vertices);

    ~Grafo() = default;

    void insereAresta(TipoValorVertice*v1, TipoValorVertice *v2,
                      TipoPeso *peso);

    bool existeAresta(TipoValorVertice v1, TipoValorVertice v2);

    bool listaAdjVazia(TipoValorVertice *v);

    TipoApontador primeiroListaAdj(TipoValorVertice *v);

    void proxAdj(TipoValorVertice *v, TipoValorVertice *adj, TipoPeso *p,
                      TipoApontador *prox, bool *fimListaAdj);

    void retiraAresta( TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *p);

    void imprimeGrafo();
};
```

Tipos auxiliares e constantes

- `MAX_VERTICES`: *limita o tamanho físico máximo para a matriz de adjacência.*
- `TipoValorVertice`: *abstrai o tipo int para definir valores de vértice.*
- `TipoPeso`: *abstrai o tipo int para definir o peso de uma aresta.*
- `TipoApontador`: *abstrai o tipo int para apontar valores para um vértice.*

Atributos privados

Os atributos armazenam o estado interno do objeto.

- `Matriz`: *armazena as arestas e seus pesos da matriz de adjacência.*
- `N_Vertices`: *armazena o número de vértices.*
- `N_Arestas`: *armazena o número de arestas.*

Métodos públicos

Construtor `Grafo()`
Finalidade: Recebe o número de vértices e inicializa um grafo com nenhuma aresta.
Parâmetros:

- `numero_vertices`: *número de vértices que se deseja inserir no grafo.*

Retorno: construtores não possuem valor de retorno.
Funcionamento: Utiliza dois laços para percorrer o atributo Matriz, atribuindo 0 a cada índice Matriz[i][j] (nenhum peso de aresta).

```
Grafo::Grafo(int numero_vertices)
    : N_Vertices(numero_vertices), N_Arestas(0){
    for (int i = 0; i < numero_vertices; i++){
        for (int j = 0; j < numero_vertices; j++){
            Matriz[i][j] = 0;
        }
    }
}
```

Método `insereAresta()`
Finalidade: Inserir uma aresta (peso) entre dois vértices.
Parâmetros:

- `v1`: *vértice de origem.*
- `v2`: *vértice de destino.*
- `peso`: *peso da aresta.*

Retorno: A função não possui retorno.
Funcionamento: A função valida os parâmetros, rejeita pesos menores ou iguais a zero e atribui o peso informado à posição correspondente da Matriz. Caso a posição ainda não contenha uma aresta, incrementa o contador de arestas.

```
void Grafo::insereAresta(TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *peso){
    if (v1 == nullptr || v2 == nullptr || peso == nullptr ||
        *v1 < 0 || *v1 >= N_Vertices ||
        *v2 < 0 || *v2 >= N_Vertices || *peso <= 0){
        return;
    }
    if (this->Matriz[*v1][*v2] == 0){
        ++N_Arestas;
    }
    this->Matriz[*v1][*v2] = *peso;
}
```

Método `existeAresta()`
Finalidade: Verifica se existe uma aresta entre dois pontos quaisquer.
Parâmetros:

- `v1`: *vértice de origem.*
- `v2`: *vértice de destino.*

Retorno: a função retorna *true* caso exista uma aresta entre os vértices (peso > 0). Caso contrário, retorna *false*. O grafo considera apenas pesos positivos.
Funcionamento: A função acessa a Matriz no índice de parâmetro e verifica se o peso é maior que 0.

```
bool Grafo::existeAresta(TipoValorVertice v1, TipoValorVertice v2){
    if (this->Matriz[v1][v2] > 0){
        return true;
    }
    return false;
}
```

Método `listaAdjVazia()`
Finalidade: Verifica se há alguma adjacência no vértice em questão.
Parâmetros:

- `v`: *vértice a ser verificado.*

Retorno: a função retorna *true* caso não haja nenhuma adjacência. Caso contrário, retorna *false*.
Funcionamento: A função utiliza um laço para percorrer todas as colunas da linha informada no parâmetro.

```
bool Grafo::listaAdjVazia(TipoValorVertice *v){
    bool vazio = true;
    int j = 0;
    while (j < this->N_Vertices && vazio){
        if (this->Matriz[*v][j] > 0){
            return false;
        } else{
            j++;
        }
    }
    return true;
}
```

Método `primeiroListaAdj()`
Finalidade: Obtém o primeiro vértice que tem adjacência a determinado vértice.
Parâmetros:

- `v`: *vértice a ser verificado.*

Retorno: Índice do primeiro vértice adjacente. Quando não há adjacência, retorna o número de vértices, pois percorreu toda a matriz.
Funcionamento: A função apresenta exatamente o mesmo comportamento do método ***listaAdjVazia()***, mas, ao final, retorna o índice usado para percorrer as colunas. Caso percorra toda a matriz, também exibe uma mensagem de que a lista de adjacência para este vértice está vazia.

```
TipoApontador Grafo::primeiroListaAdj(TipoValorVertice *v){
    TipoApontador j = 0;
    bool vazio = true;

    while(j < this->N_Vertices && vazio){
        if (this->Matriz[*v][j] > 0){
```

```
            vazio = false;
        } else{
            j++;
        }
    }

    if(j == this->N_Vertices){
        printf("ERRO: Lista adjacência vazia!\n");
    }

    return j;
}
```

Método `proxAdj()`
Finalidade: Obtém uma adjacência e avança o apontador da lista.
Parâmetros:

- `v`: *vértice cuja lista de adjacência será percorrida.*
- $\texttt{adj}_{\text{out}}$: *recebe o vértice adjacente encontrado.*
- $\texttt{p}_{\text{out}}$: *Recebe o peso da aresta entre 'v' e 'adj'.*
- $\texttt{prox}_{\text{in,out}}$: *Recebe a posição da próxima adjacência a ser consultada.*
- $\texttt{fimListaAdj}_{\text{out}}$: *Recebe true quando não existem mais adjacências a consultar e false caso ainda existam.*

Retorno: A função não possui retorno. Alguns valores de parâmetros são modificados durante sua execução.
Funcionamento: A função obtém a adjacência indicada pelo apontador *prox*, armazenando em *adj* o vértice adjacente e em *p* o peso da aresta correspondente. Em seguida, avança até a próxima posição com uma aresta e informa, por meio de *fimListaAdj*, se o final da lista de adjacências foi alcançado.

```
void Grafo::proxAdj(TipoValorVertice *v, TipoValorVertice *adj, TipoPeso *p,
                    TipoApontador *prox, bool *fimListaAdj){
    *adj = *prox;
    *p = this->Matriz[*v][*prox];
    (*prox)++;

    while(*prox < this->N_Vertices && this->Matriz[*v][*prox] == 0){
        (*prox)++;
    }

    if (*prox == this->N_Vertices){
        *fimListaAdj = true;
    } else{
        *fimListaAdj = false;
    }
}
```

Método `retiraAresta()`
Finalidade: Remove uma aresta entre dois vértices e retorna o peso que tinha.
Parâmetros:

- `v1`: *vértice de origem.*
- `v2`: *vértice de destino.*
- $\texttt{p}_{\text{out}}$: *recebe o peso da aresta removida.*

Retorno: A função não possui retorno. Um valor de parâmetro é modificado durante sua execução.
Funcionamento: A função acessa a matriz no índice Matriz[v1][v2], atribui o valor do peso desta aresta a *p* e em seguida atribui o valor 0 a este índice.

```
void Grafo::retiraAresta( TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *p){
    if (v1 == nullptr || v2 == nullptr || p == nullptr ||
        *v1 < 0 || *v1 >= N_Vertices ||
        *v2 < 0 || *v2 >= N_Vertices ||
        this->Matriz[*v1][*v2] == 0){
        printf("Aresta não existe!\n");
    } else{
        *p = this->Matriz[*v1][*v2];
        this->Matriz[*v1][*v2] = 0;
        --N_Arestas;
    }
}
```

Método `imprimeGrafo()`
Finalidade: Imprime a matriz de adjacência do grafo, com os pesos de cada aresta.
Parâmetros: A função não possui parâmetros.
Retorno: A função não possui retorno.
Funcionamento: A função imprime primeiro os índices das colunas. Em seguida, percorre com dois laços a matriz de adjacência, antes de começar a imprimir cada peso na linha, o laço imprime o índice atual, para formatar a tabela da matriz de adjacência.

```
void Grafo::imprimeGrafo(){
    printf("   ");
    for (int i = 0; i < this->N_Vertices; i++){
        printf("%3d", i);
    }
    printf("\n");

    for (int i = 0; i < this->N_Vertices; i++){
        printf("%3d", i);
        for (int j = 0; j < this->N_Vertices; j++){
            printf("%3d", this->Matriz[i][j]);
        }
        printf("\n");
    }
    cout << endl;
}
```

Grafo por Lista de Adjacência

Em uma representação de grafos utilizando lista de adjacência, podemos manter, para cada vértice do grafo, uma lista de todos os outros vértices com os quais ele tem uma aresta [2].

Implementação

Definição de classes auxiliares e seus métodos

Descrição: Para o funcionamento do grafo por lista de adjacência, foi necessária a criação de classes auxiliares. O grafo consiste em uma "lista de listas", e cada uma dessas listas é composta por células que, cada uma delas armazena um item (que armazena o peso e o vértice) e o ponteiro para a próxima célula. **Arquivo:** `grafo_lista.hpp`

```
class TipoItem
{
public:
    TipoValorVertice Vertice;
    TipoPeso Peso;
    // Definição de seu construtor
    TipoItem(TipoValorVertice v = 0, TipoPeso p = 0): Vertice(v), Peso(p){}
```

```
};


class TipoCelula {
public:
    TipoItem item;
    TipoCelula* prox;
    // Definição de seu construtor
    TipoCelula(const TipoItem& item = TipoItem(), TipoCelula* prox = nullptr)
        : item(item), prox(prox){}
};

class TipoLista{
public:
    TipoApontador Primeiro;
    TipoApontador Ultimo;
    TipoLista();
    ~TipoLista();
    bool vazia() const;
    void insere(const TipoItem& i);
    void insere(const TipoItem& i, TipoApontador l);
    bool retira(TipoApontador p, TipoItem *i);
    void imprime() const;
};
```

Tipos auxiliares e constantes

- `MAX_VERTICES`: *limita o tamanho físico máximo para as listas de adjacência.*
- `TipoValorVertice`: *abstrai o tipo int para definir valores de vértice.*
- `TipoPeso`: *abstrai o tipo int para definir o peso de uma aresta.*
- `TipoApontador`: *abstrai o ponteiro utilizado para apontar para uma célula.*

Métodos da classe TipoLista

Construtor `TipoLista()`
Finalidade: Constrói uma lista vazia.
Parâmetros: A função não possui parâmetros.
Retorno: construtores não possuem valor de retorno.
Funcionamento: Apenas utiliza o operador *new* para inicializar a lista.

```
TipoLista::TipoLista()
    : Primeiro(new TipoCelula()), Ultimo(Primeiro){}
```

Método `FLVazia()`
Finalidade: Torna a lista vazia.
Parâmetros: A função não possui parâmetros.
Retorno: A função não possui retorno.
Funcionamento: Percorre toda a lista, deletando a célula atual a cada iteração.

```
void TipoLista::FLVazia(){
    TipoApontador atual = Primeiro->prox;
    while (atual != nullptr){
        TipoApontador proxima = atual->prox;
        delete atual;
        atual = proxima;
    }
    Primeiro->prox = nullptr;
    Ultimo = Primeiro;
}
```

Método `vazia()`

Finalidade: Verifica se a lista está vazia.

Parâmetros: A função não possui parâmetros.

Retorno: a função retorna *true* caso a lista esteja vazia. Caso contrário, retorna *false*.

Funcionamento: A função apenas verifica se a última célula é igual a primeira, pois se a afirmação é verdadeira é porque não há elementos.

```
bool TipoLista::vazia() const {
    return Primeiro == Ultimo;
}
```

Método `insere()`

Finalidade: Insere elementos na lista.

Nota: Utiliza de sobrecarga para definir se irá inserir no final da lista ou em uma posição específica.

Parâmetros:

- `i`: *item a ser inserido.*
- `l`: *posição a ser inserida (opcional - utilizada apenas quando deseja-se escolher a posição da inserção).*

Retorno: A função não possui retorno.

Funcionamento: A função utiliza o apontador como índice para criar uma nova célula na posição, apontando a anterior (posição) para esta nova e apontando a nova para a próxima, que era da anterior. Em caso de a posição ser a última, aponta o valor `Ultimo` da lista para a nova célula.

```
void TipoLista::insere(const TipoItem& i){
    insere(i, Ultimo);
}

void TipoLista::insere(const TipoItem& i, TipoApontador l){
    if (l == nullptr){
        return;
    }
    TipoApontador nova = new TipoCelula(i, l->prox);
    l->prox = nova;
    if (l == Ultimo){
        Ultimo = nova;
    }
}
```

Método `retira()`

Finalidade: Retira elementos da lista.

Parâmetros:

- `p`: *elemento anterior ao que será removido.*
- `i`: *receberá durante a execução o item que foi removido, contendo seu peso e seu vértice.*

Retorno: a função retorna *true* caso a remoção seja bem-sucedida. Caso contrário, retorna *false*.

Funcionamento: A função cria um auxiliar que recebe o próximo de p, altera i para receber os atributos do item que será removido, e deleta-o. A função também verifica caso o item a ser removido seja o último da lista, para alterar o apontador de último.

```
bool TipoLista::retira(TipoApontador p, TipoItem *i){
    if (p == nullptr || p->prox == nullptr || i == nullptr){
        return false;
    }
    TipoApontador q = p->prox;
    *i = q->item;
```

```
    p->prox = q->prox;
    if (q == Ultimo){
        Ultimo = p;
    }
    delete q;
    return true;
}
```

Método `imprime()`
Finalidade: Imprime a lista.
Parâmetros: A função não possui parâmetros.
Retorno: A função não possui retorno.
Funcionamento: Percorre a lista imprimindo cada vértice e peso.

```
void TipoLista::imprime() const {
    TipoApontador aux = Primeiro->prox;
    while (aux != nullptr){
        printf("%3d (%d) ", aux->item.Vertice, aux->item.Peso);
        aux = aux->prox;
    }
    putchar('\n');
}
```

Definição de classe TipoGrafo

Descrição: Define o grafo, contendo as listas de células, o número de vértices e arestas. **Arquivo:** `grafo_lista.hpp`

```
class TipoGrafo
{
public:
    TipoLista Adj[MAX_VERTICES + 1];
    TipoValorVertice N_Vertices;
    short N_Arestas;

    TipoGrafo(TipoValorVertice n);

    void FGVazio();

    void insereAresta(TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *p);

    bool existeAresta(TipoValorVertice v1, TipoValorVertice v2);

    bool listaAdjVazia(TipoValorVertice *v);

    TipoApontador primeiroListaAdj(TipoValorVertice *v);

    void proxAdj(TipoValorVertice *v, TipoValorVertice *adj, TipoPeso *p, TipoApontador                    *pr

    void retiraAresta(TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *p);

    void liberaGrafo();

    void imprimeGrafo();
};
```

Atributos da classe Os atributos armazenam o estado interno do objeto.

- `Adj`: *vetor que armazena as listas de adjacência.*
- `N_Vertices`: *armazena o número de vértices.*
- `N_Arestas`: *armazena o número de arestas.*

Métodos da classe TipoGrafo

Método `TipoGrafo()`
Finalidade: Constrói objeto TipoGrafo.
Parâmetros:

- `n`: *número de vértices que o grafo terá.*

Retorno: construtores não possuem valor de retorno.
Funcionamento: Atribui o número de vértices do parâmetro ao N_Vertices do objeto.

```
1  TipoGrafo::TipoGrafo(TipoValorVertice n)
2      : N_Vertices(n), N_Arestas(0) {
3      if (n < 0 || n > MAX_VERTICES) {
4          N_Vertices = 0;
5      }
6  }
```

Método `FGVazio()`
Finalidade: Torna o grafo vazio.
Parâmetros: A função não possui parâmetros.
Retorno: A função não possui retorno.
Funcionamento: Percorre o vetor de listas de adjacência, e limpa cada uma delas.

```
1  void TipoGrafo::FGVazio(){
2      for (int i = 0; i < N_Vertices; i++){
3          Adj[i].FLVazia();
4      }
5      N_Arestas = 0;
6  }
```

Método `insereAresta()`
Finalidade: Insere uma aresta no grafo.
Parâmetros:

- `v1`: *vértice de origem.*
- `v2`: *vértice de destino.*
- `p`: *peso da aresta.*

Retorno: A função não possui retorno.
Funcionamento: Insere na lista do vértice seu destino com seu peso, através do método *insere()* do TipoLista.

```
1  void TipoGrafo::insereAresta(TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *p) {
2      if (v1 == nullptr || v2 == nullptr || p == nullptr ||
3          *v1 < 0 || *v1 >= N_Vertices ||
4          *v2 < 0 || *v2 >= N_Vertices) {
5          return;
6      }
7      Adj[*v1].insere(TipoItem(*v2, *p));
8      ++N_Arestas;
9  }
```

Método `existeAresta()`
Finalidade: Verifica se há uma aresta entre dois vértices.
Parâmetros:

- `v1`: *vértice de origem.*
- `v2`: *vértice de destino.*

Retorno: a função retorna *true* caso exista uma aresta entre os vértices. Caso contrário, retorna *false*.
Funcionamento: Percorre a lista de adjacência do vértice até encontrar o possível vértice de conexão.

```
bool TipoGrafo::existeAresta(TipoValorVertice v1, TipoValorVertice v2){
    if (v1 < 0 || v1 >= N_Vertices || v2 < 0 || v2 >= N_Vertices){
        return false;
    }
    for (TipoApontador aux = Adj[v1].Primeiro->prox;
         aux != nullptr;
         aux = aux->prox) {
        if (aux->item.Vertice == v2){
            return true;
        }
    }
    return false;
}
```

Método `listaAdjVazia()`
Finalidade: Verifica se a lista de adjacência para o vértice está vazia.
Parâmetros:

- `v`: *vértice a ser verificado.*

Retorno: a função retorna *true* caso não exista nenhuma adjacência para o vértice. Caso contrário, retorna *false*.
Funcionamento: Chama a função de verificação se a lista está vazia para o índice do vértice.

```
bool TipoGrafo::listaAdjVazia(TipoValorVertice *v){
    if (v == nullptr || *v < 0 || *v >= N_Vertices){
        return true;
    }
    return Adj[*v].vazia();
}
```

Método `primeiroListaAdj()`
Finalidade: Verifica qual é a primeira aresta da lista de adjacência do vértice.
Parâmetros:

- `v`: *vértice a ser verificado.*

Retorno: o primeiro vértice adjacente ao vértice verificado.
Funcionamento: Busca o próximo do primeiro vértice da lista (a primeira adjacência).

```
TipoApontador TipoGrafo::primeiroListaAdj(TipoValorVertice *v){
    if (v == nullptr || *v < 0 || *v >= N_Vertices){
        return nullptr;
    }
    return Adj[*v].Primeiro->prox;
}
```

Método `proxAdj()`

Finalidade: Obtém uma adjacência e avança o apontador da lista.

Parâmetros:

- `v`: *vértice cuja lista de adjacência será percorrida.*
- $\texttt{adj}_{\text{out}}$: *recebe o vértice adjacente encontrado.*
- $\texttt{p}_{\text{out}}$: *Recebe o peso da aresta entre 'v' e 'adj'.*
- $\texttt{prox}_{\text{in,out}}$: *Recebe a posição da próxima adjacência a ser consultada.*
- $\texttt{fimListaAdj}_{\text{out}}$: *Recebe true quando não existem mais adjacências a consultar e false caso ainda existam.*

Retorno: A função não possui retorno. Alguns valores de parâmetros são modificados durante sua execução.

Funcionamento: A função obtém a adjacência indicada pelo apontador *prox*, armazenando em *adj* o vértice adjacente e em *p* o peso da aresta correspondente. Em seguida, avança até a próxima posição com uma aresta e informa, por meio de *fimListaAdj*, se o final da lista de adjacências foi alcançado.

```
1  void TipoGrafo::proxAdj(TipoValorVertice *v, TipoValorVertice *adj, TipoPeso *p,
2      if (v == nullptr || adj == nullptr || p == nullptr ||
3          prox == nullptr || fimListaAdj == nullptr || *prox == nullptr){
4          *fimListaAdj = true;
5          return;
6      }
7      *adj = (*prox)->item.Vertice;
8      *p = (*prox)->item.Peso;
9      *prox = (*prox)->prox;
10     *fimListaAdj = (*prox == nullptr);
11 }
```

Método `retiraAresta()`

Finalidade: Remove uma aresta entre dois vértices e coloca em '*p' o peso que a aresta tinha.

Parâmetros:

- `v1`: *vértice de origem.*
- `v2`: *vértice de destino.*
- `p`: *receberá o peso da aresta removida.*

Retorno: A função não possui retorno, mas altera uma variável durante sua execução.

Funcionamento: Percorre a lista até encontrar os dois vértices que estão ligados pela aresta e remove o item da lista.

```
1  void TipoGrafo::retiraAresta(TipoValorVertice *v1, TipoValorVertice *v2, TipoPeso *p){
2      if (v1 == nullptr || v2 == nullptr || p == nullptr ||
3          *v1 < 0 || *v1 >= N_Vertices ||
4          *v2 < 0 || *v2 >= N_Vertices){
5          return;
6      }
7      TipoApontador anterior = Adj[*v1].Primeiro;
8      while (anterior->prox != nullptr){
9          if (anterior->prox->item.Vertice == *v2){
10             TipoItem removido;
11             if (Adj[*v1].retira(anterior, &removido)){
12                 *p = removido.Peso;
13                 --N_Arestas;
14             }
15             return;
16         }
17         anterior = anterior->prox;
18     }
19 }
```

Método `liberaGrafo()`
Finalidade: Limpa o grafo.
Parâmetros: A função não possui parâmetros.
Retorno: A função não possui retorno.
Funcionamento: Chama a função que torna o grafo vazio.

```
void TipoGrafo::liberaGrafo(){
    FGVazio();
}
```

Método `imprimeGrafo()`
Finalidade: Imprime as listas de adjacência de cada vértice.
Parâmetros: A função não possui parâmetros.
Retorno: A função não possui retorno.
Funcionamento: Para cada vértice, imprime a sua lista de adjacência.

```
void TipoGrafo::imprimeGrafo(){
    for (int i = 0; i < N_Vertices; i++){
        printf("Vertice%2d: ", i);
        Adj[i].imprime();
    }
}
```

Referências

[1] MATRIZ DE ADJACÊNCIA. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2017. Disponível em: `https://wikipedia.org`. Acesso em: 6 jan. 2017.

[2] LISTA DE ADJACÊNCIA. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: `https://wikipedia.org`. Acesso em: 20 mar. 2018.